

# Cloud Computing Basics Cheat Sheet

15 tables · 108 concepts · Updated 2026-04-20 · [cheatgrid.com](#)

**Cloud computing** is the on-demand delivery of IT resources over the Internet with **pay-as-you-go pricing**, fundamentally changing how organizations access and manage technology infrastructure. Rather than investing in and maintaining physical data centers, organizations can provision **compute, storage, databases, networking, and other services** instantly from cloud providers like AWS, Microsoft Azure, and Google Cloud Platform. The key mental model: cloud computing shifts IT from a capital expenditure (buying servers) to an operational expenditure (renting capacity), enabling unprecedented **agility, scalability, and cost efficiency** while letting teams focus on innovation rather than infrastructure management.

CONTENTS

|                                             |   |
|---------------------------------------------|---|
| Service Models                              | 1 |
| Deployment Models                           | 2 |
| Essential Characteristics (NIST SP 800-145) | 3 |
| Core Benefits                               | 3 |
| Global Infrastructure                       | 4 |
| Computing Resources                         | 4 |
| Storage Services                            | 5 |
| Networking Concepts                         | 5 |
| Database Services                           | 6 |
| Pricing Models                              | 6 |
| Security and Compliance                     | 7 |
| Migration Strategies (The 7 Rs)             | 7 |
| Management and Monitoring                   | 8 |
| Development and DevOps                      | 8 |
| Well-Architected Framework Pillars          | 9 |
| References                                  | 9 |

TABLE 1

## Service Models

Service models answer a single question: how much of the stack does the provider manage versus you? The classic trio — IaaS, PaaS, SaaS — runs from renting raw virtual machines to consuming finished applications in a browser, with each step handing off more of the underlying layers. The newer "as-a-service" rows extend the same idea to functions, containers, databases, and even AI, so you can outsource exactly the layer that isn't your core problem.

| Model                              | Example                                                     | Description                                                                                                                                                                                                   |
|------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Infrastructure as a Service (IaaS) | AWS EC2, Azure Virtual Machines, Google Compute Engine      | Provides <b>virtualized computing resources</b> over the internet — you manage OS, runtime, middleware, and applications while the provider manages physical infrastructure, networking, and storage hardware |
| Platform as a Service (PaaS)       | AWS Elastic Beanstalk, Azure App Service, Google App Engine | Offers a complete <b>development and deployment environment</b> in the cloud — handles infrastructure, OS, middleware, and runtime so developers focus solely on code and data                                |
| Software as a Service (SaaS)       | Microsoft 365, Salesforce, Google Workspace, Dropbox        | Delivers <b>fully managed applications</b> over the internet — users access via browser with zero infrastructure management, automatic updates, and subscription pricing                                      |

| Model                                         | Example                                                         | Description                                                                                                                                                                                    |
|-----------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Function as a Service (FaaS)</b>           | AWS Lambda, Azure Functions, Google Cloud Functions             | Enables <b>event-driven code execution</b> without managing servers — pay only for actual compute time in milliseconds, with automatic scaling from zero to thousands of concurrent executions |
| <b>Container as a Service (CaaS)</b>          | Amazon ECS, Azure Container Instances, Google Kubernetes Engine | Offers <b>container orchestration and management</b> — deploy, scale, and manage containerized applications without managing the underlying infrastructure                                     |
| <b>Database as a Service (DBaaS)</b>          | Amazon RDS, Azure SQL Database, Google Cloud SQL                | Provides <b>managed database services</b> with automated backups, patching, and scaling — eliminates database administration overhead while maintaining high availability                      |
| <b>Backend as a Service (BaaS)</b>            | Firebase, AWS Amplify, Azure Mobile Apps                        | Delivers <b>ready-to-use backend functionality</b> like authentication, databases, and APIs — primarily for mobile and web applications with minimal backend coding                            |
| <b>Disaster Recovery as a Service (DRaaS)</b> | Zerto, Azure Site Recovery, AWS Elastic Disaster Recovery       | Provides <b>cloud-based disaster recovery</b> — replicates and hosts physical or virtual servers to provide failover in the event of a natural or man-made disaster                            |
| <b>Network as a Service (NaaS)</b>            | Cisco Meraki, Megaport, Cloudflare Magic WAN                    | Delivers <b>virtualized networking services</b> over the cloud — replaces traditional hardware-centric networking with on-demand, software-defined connectivity                                |
| <b>AI as a Service (AlaaS)</b>                | Amazon SageMaker, Google Vertex AI, Azure AI Services           | Provides <b>managed AI and ML platforms</b> — offers pre-trained models, training infrastructure, and inference APIs without building ML pipelines from scratch                                |

TABLE 2

## Deployment Models

Where the infrastructure lives and who shares it is a separate decision from which service model you pick. Public cloud means shared, provider-owned infrastructure at maximum scale; private cloud is dedicated to one organization for tighter control; and hybrid and multi-cloud blend approaches to balance cost, compliance, and vendor lock-in. The sovereign and community rows reflect the growing pressure to keep data inside specific legal borders.

| Model                  | Example                                             | Description                                                                                                                                                                                                    |
|------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Public Cloud</b>    | AWS, Microsoft Azure, Google Cloud Platform         | Infrastructure owned and operated by <b>third-party cloud providers</b> — resources shared among multiple organizations (multi-tenancy) with internet access, offering maximum scalability and cost efficiency |
| <b>Private Cloud</b>   | VMware Cloud, OpenStack on-premises, Azure Stack    | Dedicated infrastructure used <b>exclusively by one organization</b> — can be hosted on-premises or by a third party, providing greater control, security, and compliance at higher cost                       |
| <b>Hybrid Cloud</b>    | AWS Outposts, Azure Arc, Google Anthos              | Combines <b>public and private clouds</b> with orchestration between them — allows workloads to move between environments based on computing needs, compliance, or cost optimization                           |
| <b>Multi-Cloud</b>     | Using AWS + Azure + GCP simultaneously              | Strategy using <b>multiple cloud providers</b> concurrently — avoids vendor lock-in, leverages best-of-breed services, and provides redundancy, but increases management complexity                            |
| <b>Community Cloud</b> | AWS GovCloud, Azure Government                      | Shared infrastructure for <b>organizations with common concerns</b> — typically regulatory compliance, security requirements, or industry-specific needs like healthcare or finance                            |
| <b>Sovereign Cloud</b> | T-Systems Sovereign Cloud, Google Distributed Cloud | Infrastructure ensuring <b>data stays within national borders</b> under local jurisdiction — addresses data sovereignty and regulatory requirements increasingly mandated by governments in 2026               |

TABLE 3

## Essential Characteristics (NIST SP 800-145)

These five traits are NIST's official definition of what makes something genuinely "cloud" rather than just hosting — and they show up constantly on certification exams. Together they describe a system you can provision yourself, reach from any device, that pools resources across many tenants, scales elastically with demand, and meters everything so you pay only for what you use.

| Characteristic                | Example                                                 | Description                                                                                                                                                                         |
|-------------------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>On-Demand Self-Service</b> | Launching a VM via AWS console without calling support  | Users can <b>provision resources automatically</b> without human interaction with the service provider — typically through web consoles, APIs, or command-line tools                |
| <b>Broad Network Access</b>   | Accessing cloud applications from laptop, phone, tablet | Resources accessible over the <b>network via standard protocols</b> — supports heterogeneous client platforms including mobile phones, tablets, laptops, and workstations           |
| <b>Resource Pooling</b>       | Multiple tenants sharing same physical servers          | Provider's computing resources <b>serve multiple customers</b> using multi-tenancy — physical and virtual resources dynamically assigned based on demand with location independence |
| <b>Rapid Elasticity</b>       | Auto-scaling web servers during traffic spike           | Resources can be <b>scaled up or down quickly</b> and automatically — often seamlessly to the user, appearing unlimited and available in any quantity at any time                   |
| <b>Measured Service</b>       | Pay-per-GB storage, pay-per-hour compute                | Resource usage is <b>monitored, controlled, and reported</b> — provides transparency for both provider and consumer, enabling pay-per-use or charge-per-use business models         |

TABLE 4

## Core Benefits

This is the "why bother" list — the concrete payoffs that drive organizations off their own data centers. The headline is the shift from capital expense to operational expense, but the rest of the table is just as important: scalability and elasticity to match capacity to demand, agility to ship faster, and the global reach, reliability, and security you'd struggle to build alone. A handy distinction to hold onto is scalability (the ability to grow) versus elasticity (growing and shrinking automatically).

| Benefit                             | Example                                                         | Description                                                                                                                                                                                      |
|-------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cost Savings (CapEx to OpEx)</b> | Trading \$100K data center investment for \$1K/month cloud bill | Eliminates <b>capital expenses</b> for hardware and data centers — converts to variable operational expenses with pay-only-for-what-you-use pricing and economies of scale                       |
| <b>Scalability</b>                  | Adding 100 servers during holiday shopping season               | Ability to <b>increase or decrease resources</b> to handle workloads — can scale vertically (bigger machines) or horizontally (more machines) based on demand                                    |
| <b>Elasticity</b>                   | Auto-scaling from 2 to 200 instances and back to 2              | <b>Automatic scaling</b> of resources based on current demand — scales both up and down dynamically, ensuring optimal resource utilization and cost efficiency                                   |
| <b>Agility</b>                      | Deploying new environment in minutes vs weeks                   | <b>Faster time-to-market</b> for new capabilities — provision resources in minutes, experiment rapidly, and iterate quickly without long procurement cycles                                      |
| <b>Global Reach</b>                 | Deploying application to 30+ geographic regions in minutes      | Deploy applications in <b>multiple geographic locations worldwide</b> — reduces latency for end users and enables disaster recovery across continents                                            |
| <b>Reliability</b>                  | 99.99% uptime SLA with automatic failover                       | Enhanced availability through <b>multiple redundant sites</b> — providers offer backup, disaster recovery, and data replication across geographically distributed infrastructure                 |
| <b>Security</b>                     | Built-in DDoS protection, encryption, compliance certifications | Benefit from <b>provider's massive security investments</b> — includes physical security, network security, encryption, compliance certifications, and continuous monitoring at enterprise scale |
| <b>Performance</b>                  | Access to latest CPUs, GPUs, and high-speed networking          | Use <b>cutting-edge hardware</b> without capital investment — providers continually upgrade infrastructure with latest processors, storage, and networking technologies                          |

| Benefit        | Example                                                                  | Description                                                                                                                                                                       |
|----------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sustainability | AWS targets 100% renewable energy by 2025, Azure carbon-negative by 2030 | Cloud providers invest heavily in <b>energy-efficient data centers</b> and renewable energy — migrating to cloud can reduce carbon footprint by up to 80% compared to on-premises |

TABLE 5  
Global Infrastructure

Cloud providers organize their physical footprint into a nested geography that you'll deploy into deliberately. Regions are broad geographic areas, availability zones are the isolated data centers inside them that let you survive a single failure, and edge and local zones push compute closer to users for lower latency. Data sovereignty is the legal overlay — the rules dictating which region your data is even allowed to sit in.

| Concept                | Example                                                                        | Description                                                                                                                                                               |
|------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Region                 | <code>us-east-1</code> (N. Virginia),<br><code>eu-west-1</code> (Ireland)      | A <b>geographic area</b> containing multiple isolated data center clusters — choose regions based on latency, compliance, and data residency requirements                 |
| Availability Zone (AZ) | <code>us-east-1a</code> , <code>us-east-1b</code> ,<br><code>us-east-1c</code> | <b>Isolated data center(s)</b> within a region — connected via low-latency links, designed so a failure in one AZ does not affect others                                  |
| Edge Location          | CloudFront PoP in Tokyo caching static content                                 | <b>CDN endpoint</b> closer to end users — caches content at 400+ points of presence worldwide for lower latency content delivery                                          |
| Local Zone             | AWS Local Zone in Los Angeles for low-latency gaming                           | <b>Extension of a region</b> placed in a metro area — provides single-digit millisecond latency for latency-sensitive applications                                        |
| Data Sovereignty       | GDPR requiring EU citizen data stays in EU                                     | Legal requirement that data is <b>subject to laws of the country</b> where it is stored — increasingly important as regulations like GDPR, NIS2, and DORA tighten in 2026 |

TABLE 6  
Computing Resources

These are the actual ways you run code in the cloud, arranged loosely from heaviest to lightest. Virtual machines give you a full OS to control, containers package just your app and its dependencies for faster, denser deployment, and serverless functions strip it down to running code on a trigger with no server to manage. The specialty rows — bare metal, GPU, and spot instances — handle performance-critical, accelerated, or cost-sensitive interruptible workloads.

| Type                       | Example                                                                                     | Description                                                                                                                                                                  |
|----------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Virtual Machines           | <code>aws ec2 run-instances --image-id ami-12345678</code>                                  | <b>Virtualized servers</b> running complete operating systems — provide full OS control with customizable CPU, memory, and storage configurations                            |
| Containers                 | Docker containers running on Amazon ECS or Kubernetes                                       | <b>Lightweight virtualization</b> packaging application and dependencies — share OS kernel for faster startup and higher density than VMs                                    |
| Serverless Functions       | <code>def lambda_handler(event, context):</code><br><code>return {'statusCode': 200}</code> | <b>Event-triggered code execution</b> without server management — scales automatically from zero, charges only for actual execution time                                     |
| Bare Metal Instances       | AWS i3.metal, Azure Bare Metal, GCP C3 bare metal                                           | <b>Physical servers</b> without virtualization layer — provides maximum performance for workloads sensitive to virtualization overhead or requiring specific hardware access |
| GPU Instances              | NVIDIA H100 instances for LLM training                                                      | Specialized compute with <b>graphics processing units</b> — accelerates machine learning, scientific computing, graphics rendering, and video encoding workloads             |
| Spot/Preemptible Instances | Bidding for unused EC2 capacity at 90% discount                                             | <b>Unused capacity</b> at significantly reduced prices — can be interrupted with short notice, ideal for fault-tolerant batch processing and stateless workloads             |

TABLE 7

## Storage Services

Cloud storage isn't one thing — the type you choose depends on how your data is accessed. Object storage holds unstructured files like media and backups at near-infinite scale, block storage gives VMs fast low-latency volumes for databases, and file storage offers shared network filesystems multiple machines can mount at once. Archive storage trades retrieval speed for the lowest cost, while ephemeral storage is fast but vanishes when the instance stops.

| Type                     | Example                                              | Description                                                                                                                                                                                         |
|--------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Object Storage</b>    | Amazon S3, Azure Blob Storage, Google Cloud Storage  | Stores data as <b>discrete objects with metadata</b> — highly scalable and durable (99.999999999% durability), accessed via HTTP APIs, ideal for unstructured data like images, videos, and backups |
| <b>Block Storage</b>     | Amazon EBS, Azure Managed Disks, GCP Persistent Disk | Provides <b>raw storage volumes</b> attached to VMs — low-latency persistent storage for databases and applications requiring direct block-level access                                             |
| <b>File Storage</b>      | Amazon EFS, Azure Files, Google Filestore            | Offers <b>network-attached file systems</b> — supports concurrent access from multiple instances using standard file protocols like NFS or SMB                                                      |
| <b>Archive Storage</b>   | Amazon S3 Glacier, Azure Archive Storage             | <b>Long-term cold storage</b> at lowest cost — optimized for data accessed rarely, with retrieval times from minutes to hours                                                                       |
| <b>Ephemeral Storage</b> | EC2 instance store volumes                           | <b>Temporary storage</b> physically attached to host — high performance but data lost when instance stops, suitable for temporary caches and buffers                                                |

TABLE 8

## Networking Concepts

The VPC is the foundation of cloud networking — your own isolated, software-defined network where you control IP ranges, subnets, and routing. The rest of the table builds outward from there: subnets segment it into public and private zones, load balancers and CDNs distribute traffic, and gateways and connectivity options (VPN, Direct Connect, Transit Gateway, Private Link) govern how traffic flows in, out, and between networks securely.

| Concept                                | Example                                                                        | Description                                                                                                                                               |
|----------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Virtual Private Cloud (VPC)</b>     | Isolated network: <code>10.0.0.0/16</code> with private subnets                | <b>Logically isolated network</b> within cloud provider — provides complete control over IP addressing, subnets, routing tables, and network gateways     |
| <b>Subnet</b>                          | Public subnet: <code>10.0.1.0/24</code> ,<br>Private: <code>10.0.2.0/24</code> | <b>Segment of VPC's IP address range</b> — resources in public subnets have internet access while private subnets remain isolated                         |
| <b>Load Balancer</b>                   | Distributing traffic across 10 web servers                                     | <b>Distributes incoming traffic</b> across multiple targets — improves availability and fault tolerance by routing around unhealthy instances             |
| <b>Content Delivery Network (CDN)</b>  | CloudFront caching static assets at 400+ edge locations                        | <b>Globally distributed cache servers</b> — serve content closer to end users, reducing latency and bandwidth costs for static and dynamic content        |
| <b>DNS Service</b>                     | Route 53 resolving <code>example.com</code> to load balancer IP                | <b>Domain Name System management</b> — translates domain names to IP addresses with routing policies for failover, geolocation, and latency-based routing |
| <b>Virtual Private Network (VPN)</b>   | Encrypted tunnel between on-premises and cloud VPC                             | <b>Secure encrypted connection</b> over public internet — connects on-premises networks to cloud resources with IPsec encryption                          |
| <b>Direct Connect / ExpressRoute</b>   | Dedicated 10 Gbps fiber connection to AWS                                      | <b>Private dedicated network connection</b> — bypasses internet for more reliable, lower-latency, and higher-throughput access to cloud resources         |
| <b>NAT Gateway</b>                     | Private instances accessing internet for software updates                      | Enables instances in <b>private subnets to reach the internet</b> — outbound-only connectivity that prevents unsolicited inbound connections              |
| <b>Transit Gateway</b>                 | Connecting 50 VPCs and on-premises networks through a hub                      | <b>Central hub</b> that connects multiple VPCs and on-premises networks — simplifies network architecture by replacing complex peering relationships      |
| <b>Private Link / Private Endpoint</b> | Accessing S3 from VPC without traversing public internet                       | <b>Private connectivity</b> to cloud services — keeps traffic within provider's network without exposure to the public internet                           |

TABLE 9

## Database Services

There's no universal database, so the cloud offers a managed engine for every data shape. Relational databases handle structured, transactional data with ACID guarantees; NoSQL trades that rigidity for flexible schemas and horizontal scale; and in-memory stores deliver microsecond latency for caching. The specialized rows — data warehouses, time-series, graph, and lakehouse — exist because analytics, sensor streams, relationship data, and mixed structured/unstructured workloads each query very differently.

| Type                              | Example                                                   | Description                                                                                                                                                                                      |
|-----------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Relational Databases (SQL)</b> | Amazon RDS (MySQL, PostgreSQL), Azure SQL Database        | <b>SQL databases</b> managed by provider — automated backups, patching, scaling, and high availability for ACID-compliant transactional workloads                                                |
| <b>NoSQL Databases</b>            | DynamoDB, Azure Cosmos DB, MongoDB Atlas                  | <b>Non-relational databases</b> for flexible schemas — supports key-value, document, column-family, and graph models with horizontal scaling                                                     |
| <b>In-Memory Databases</b>        | Redis, Memcached on Amazon ElastiCache                    | <b>RAM-based databases</b> for microsecond latency — ideal for caching, session management, real-time analytics, and leaderboards                                                                |
| <b>Data Warehouses</b>            | Amazon Redshift, Azure Synapse Analytics, Google BigQuery | <b>Columnar storage</b> optimized for analytics — handles petabyte-scale data with fast query performance for business intelligence and reporting                                                |
| <b>Time-Series Databases</b>      | Amazon Timestream, Azure Data Explorer                    | Specialized for <b>timestamped data</b> — efficiently stores and queries IoT sensor data, metrics, and events with time-based aggregations                                                       |
| <b>Graph Databases</b>            | Amazon Neptune, Azure Cosmos DB (Gremlin API)             | Optimized for <b>relationships between data</b> — uses nodes, edges, and properties for social networks, recommendation engines, and fraud detection                                             |
| <b>Data Lake / Lakehouse</b>      | AWS Lake Formation, Databricks Lakehouse, Azure Data Lake | Centralized repository storing <b>structured and unstructured data</b> at any scale — lakehouse architecture combines data lake flexibility with warehouse performance for both analytics and ML |

TABLE 10

## Pricing Models

How you pay for the same resource can swing the bill dramatically. On-demand is the flexible default with zero commitment, while reserved instances and savings plans trade a one-to-three-year commitment for discounts of up to 72%. Spot pricing offers the deepest cuts on spare capacity that can be reclaimed at any moment, and the free tier and data transfer rows are the two that catch newcomers out — one for what's free, the other for the egress charges that quietly add up.

| Model                                     | Example                                               | Description                                                                                                                                          |
|-------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Pay-As-You-Go (On-Demand)</b>          | \$0.10/hour for compute, \$0.023/GB-month for storage | Pay only for <b>resources actually consumed</b> — no upfront costs or long-term commitments, billed by second, hour, or GB with usage-based pricing  |
| <b>Reserved Instances / Committed Use</b> | 1-year commitment for 40% discount, 3-year for 72%    | <b>Commit to usage</b> for discounted rates — pay upfront or monthly for 1–3 year terms, saving up to 72% over on-demand pricing                     |
| <b>Savings Plans</b>                      | Commit to \$10/hour for 72% discount vs on-demand     | <b>Flexible pricing model</b> — commit to consistent usage measured in \$/hour for 1–3 years, applies automatically across eligible compute services |
| <b>Spot / Preemptible Pricing</b>         | Bidding for unused capacity at up to 90% discount     | Purchase <b>spare capacity</b> at deep discounts — prices fluctuate based on supply and demand, instances can be interrupted with 2-minute notice    |
| <b>Free Tier</b>                          | 750 hours/month EC2 t2.micro, 5 GB S3 storage         | <b>Limited free usage</b> for new accounts — typically 12 months of certain services plus always-free offerings to experiment and learn              |
| <b>Data Transfer Pricing</b>              | Free inbound, \$0.09/GB outbound to internet          | Charges for <b>data movement</b> — inbound typically free, outbound to internet charged per GB, inter-region transfer also incurs costs              |

TABLE 11

## Security and Compliance

Security in the cloud starts with the shared responsibility model — understanding the line between what the provider secures and what stays on you is the single most important concept here. The remaining rows are the controls you own on your side of that line: IAM for who can access what, encryption at rest and in transit for protecting data, MFA and zero trust for verifying identity, and security groups, KMS, and WAF for guarding traffic, keys, and applications.

| Concept                                     | Example                                                           | Description                                                                                                                                                                                                   |
|---------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Shared Responsibility Model</b>          | Provider secures infrastructure; customer secures data and config | Defines <b>who is responsible for what</b> — provider manages security <i>of</i> the cloud (physical, network, hypervisor), customer manages security <i>in</i> the cloud (data, IAM, encryption, app config) |
| <b>Identity and Access Management (IAM)</b> | Creating user with read-only S3 access policy                     | <b>Controls who can access what</b> — manages authentication and authorization with users, groups, roles, and fine-grained permissions using least-privilege principle                                        |
| <b>Encryption at Rest</b>                   | S3 bucket with AES-256 server-side encryption                     | Data <b>encrypted while stored</b> on disk — uses provider-managed or customer-managed keys to protect data from unauthorized physical access                                                                 |
| <b>Encryption in Transit</b>                | TLS 1.3 for HTTPS connections to cloud services                   | Data <b>encrypted during transmission</b> — uses SSL/TLS protocols to protect data moving between clients and cloud services or between cloud services                                                        |
| <b>Multi-Factor Authentication (MFA)</b>    | Requiring password + authenticator app token                      | <b>Additional authentication factor</b> beyond password — requires something you know (password) and something you have (device) for critical operations                                                      |
| <b>Security Groups / Firewalls</b>          | Allow port 443 from 0.0.0.0/0<br>Allow port 22 from 10.0.0.0/8    | <b>Virtual firewalls</b> controlling inbound and outbound traffic — stateful rules applied at instance level based on protocol, port, and source/destination                                                  |
| <b>Zero Trust Security</b>                  | Verifying every request regardless of network location            | <b>"Never trust, always verify"</b> security model — requires strict identity verification for every user and device, even inside the network perimeter                                                       |
| <b>Key Management Service (KMS)</b>         | Creating and rotating encryption keys automatically               | <b>Centralized encryption key management</b> — create, rotate, and control cryptographic keys used to encrypt data with audit logging                                                                         |
| <b>Web Application Firewall (WAF)</b>       | Blocking SQL injection and XSS attacks                            | <b>Application-layer protection</b> — filters malicious traffic based on rules for common web exploits before reaching applications                                                                           |
| <b>Compliance Certifications</b>            | SOC 2, ISO 27001, HIPAA, PCI DSS, GDPR                            | <b>Third-party validated compliance</b> — providers maintain certifications for industry standards and regulations, simplifying customer compliance                                                           |

TABLE 12

## Migration Strategies (The 7 Rs)

When moving existing workloads to the cloud, the "7 Rs" are the standard menu of approaches, ranging from minimal change to total reinvention. Rehosting is the fast lift-and-shift that changes nothing, replatforming makes small cloud optimizations along the way, and refactoring fully re-architects for cloud-native benefits. The rest cover the choices to repurchase a SaaS alternative, relocate at the hypervisor level, retire what's no longer needed, or simply retain it on-premises for now.

| Strategy                             | Example                                                  | Description                                                                                                                                            |
|--------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rehost (Lift and Shift)</b>       | Moving VM from VMware to AWS EC2 unchanged               | <b>Migrate without modifications</b> — move applications as-is to cloud infrastructure, fastest approach but doesn't leverage cloud-native features    |
| <b>Replatform (Lift and Reshape)</b> | Migrating database from self-managed MySQL to Amazon RDS | <b>Minor optimizations</b> during migration — make targeted cloud optimizations without changing core architecture, like switching to managed services |
| <b>Refactor (Re-architect)</b>       | Redesigning monolith as microservices with containers    | <b>Redesign for cloud-native</b> — completely rearchitect applications to leverage cloud features like auto-scaling, serverless, and managed services  |
| <b>Repurchase (Drop and Shop)</b>    | Replacing on-premises CRM with Salesforce SaaS           | <b>Switch to different product</b> — typically moving from traditional software to SaaS, abandoning existing application for cloud-native alternative  |

| Strategy                                | Example                                               | Description                                                                                                                                   |
|-----------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Relocate (Hypervisor-Level Lift)</b> | VMware Cloud on AWS moving entire vSphere environment | <b>Move at infrastructure level</b> — migrate existing VMs to cloud without purchasing new hardware, using hypervisor-level compatibility     |
| <b>Retire</b>                           | Decommissioning unused legacy application             | <b>Eliminate unnecessary systems</b> — identify and turn off applications no longer needed, reducing migration scope and costs                |
| <b>Retain</b>                           | Keeping mainframe system on-premises for now          | <b>Keep in current environment</b> — defer migration for applications not ready for cloud or requiring specialized on-premises infrastructure |

TABLE 13

## Management and Monitoring

Once workloads are live, these practices keep them healthy, affordable, and under control. Infrastructure as code makes deployments repeatable and auditable, monitoring and observability tell you what's actually happening, and auto scaling adjusts capacity to demand. Rounding it out are the disciplines that keep an estate sane at scale — backup and DR for continuity, FinOps for spend, tagging for organization, landing zones for governance, and SLAs for the uptime you're promised.

| Concept                                | Example                                                  | Description                                                                                                                                                                      |
|----------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Infrastructure as Code (IaC)</b>    | Terraform, AWS CloudFormation, Pulumi, OpenTofu          | <b>Define infrastructure in code</b> — version-controlled templates that provision and manage resources consistently, enabling repeatable and auditable deployments              |
| <b>Monitoring and Observability</b>    | CloudWatch, Datadog, Grafana tracking CPU and latency    | <b>Collect and correlate metrics, logs, and traces</b> — monitors resource utilization, application performance, and operational health with dashboards and alerting             |
| <b>Auto Scaling</b>                    | Scale from 2 to 10 instances when CPU > 70%              | <b>Automatic capacity adjustment</b> — adds or removes resources based on demand using policies (target tracking, step, scheduled), ensuring performance while controlling costs |
| <b>Backup and Disaster Recovery</b>    | Automated daily snapshots with 30-day retention          | <b>Automated data protection</b> — scheduled backups, point-in-time recovery, and cross-region replication for business continuity                                               |
| <b>Cost Management / FinOps</b>        | Budget alerts when spending exceeds \$1,000/month        | <b>Track and optimize cloud spending</b> — provides cost visibility, budgets, forecasting, rightsizing recommendations, and anomaly detection to reduce waste                    |
| <b>Resource Tagging</b>                | Environment: Production, Team: Engineering               | <b>Metadata labels</b> for resources — enables organization, cost allocation, access control, and automation based on tag key-value pairs                                        |
| <b>Landing Zone</b>                    | AWS Control Tower setting up multi-account environment   | <b>Pre-configured multi-account environment</b> with guardrails — establishes security baselines, network architecture, and governance policies for cloud adoption at scale      |
| <b>Service Level Agreements (SLAs)</b> | EC2 SLA: 99.99% monthly uptime (~4.3 min downtime/month) | <b>Contractual uptime guarantees</b> from provider — defines availability targets (e.g., three nines = 99.9%, four nines = 99.99%) and credits for breaches                      |

TABLE 14

## Development and DevOps

These are the building blocks of modern cloud-native applications and the pipelines that ship them. CI/CD automates build-test-deploy, container orchestration manages fleets of containers, and API gateways expose and protect your services. Message queues and event streaming decouple components for resilience, while artifact registries, secrets management, and service mesh handle storing builds, guarding credentials, and securing service-to-service traffic.

| Concept                | Example                                                   | Description                                                                                                                                      |
|------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CI/CD Pipelines</b> | GitHub Actions, GitLab CI, AWS CodePipeline, Azure DevOps | <b>Automated build, test, and deployment</b> — continuous integration and delivery workflows that validate and deploy code changes automatically |

| Concept                        | Example                                                        | Description                                                                                                                                                    |
|--------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Container Orchestration</b> | Kubernetes managing 100 containerized microservices            | <b>Automated container lifecycle management</b> — handles scheduling, scaling, networking, load balancing, and self-healing for containerized applications     |
| <b>API Gateway</b>             | Exposing Lambda functions as REST/HTTP APIs                    | <b>Managed API creation and management</b> — create, publish, and secure APIs at scale with authentication, throttling, rate limiting, and monitoring          |
| <b>Message Queues</b>          | Amazon SQS, Azure Service Bus, Google Pub/Sub                  | <b>Asynchronous message passing</b> — decouples components using queues for reliable message delivery between distributed systems                              |
| <b>Event Streaming</b>         | Amazon MSK (Kafka), Amazon Kinesis, Azure Event Hubs           | <b>Real-time data streaming</b> — processes high-throughput event streams for analytics, log aggregation, and event-driven architectures                       |
| <b>Artifact Registry</b>       | Amazon ECR, Azure Container Registry, Google Artifact Registry | <b>Store and manage build artifacts</b> — private registries for container images, packages, and dependencies with vulnerability scanning                      |
| <b>Secrets Management</b>      | Storing database credentials with automatic rotation           | <b>Centralized secrets storage</b> — securely stores and rotates credentials, API keys, and certificates with encryption and access control                    |
| <b>Service Mesh</b>            | Istio, Linkerd, Consul Connect on Kubernetes                   | <b>Dedicated infrastructure layer</b> for service-to-service communication — provides traffic management, observability, and security (mTLS) for microservices |

TABLE 15

## Well-Architected Framework Pillars

These six pillars are the lens cloud providers use to evaluate whether a workload is built well, and they make a useful checklist for any architecture review. Each one names a tension worth balancing — operational excellence and reliability for running smoothly and recovering from failure, security for protecting it, performance efficiency and cost optimization for doing more with less, and sustainability for minimizing environmental impact.

| Pillar                        | Example                                                      | Description                                                                                                                                                          |
|-------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Operational Excellence</b> | Automated runbooks, IaC deployments, incident response plans | Run and monitor systems to <b>deliver business value</b> — focuses on automating operations, responding to events, and continuously improving processes              |
| <b>Security</b>               | Least-privilege IAM, encryption everywhere, audit logging    | <b>Protect information and systems</b> — implement strong identity controls, enable traceability, apply security at all layers, and automate security best practices |
| <b>Reliability</b>            | Multi-AZ deployments, auto-recovery, chaos engineering       | Ensure workloads perform <b>correctly and consistently</b> — design for automatic recovery from failure, test disaster procedures, and scale horizontally            |
| <b>Performance Efficiency</b> | Right-sizing instances, using serverless, caching layers     | Use computing resources <b>efficiently as demand changes</b> — select the right resource types, monitor performance, and evolve with new technologies                |
| <b>Cost Optimization</b>      | Reserved instances, spot usage, rightsizing, idle shutdown   | Deliver value at the <b>lowest price point</b> — understand spending, select the right pricing models, and eliminate waste through continuous optimization           |
| <b>Sustainability</b>         | Choosing efficient instance types, reducing idle resources   | Minimize <b>environmental impact</b> of cloud workloads — select efficient hardware, right-size resources, and use managed services to share infrastructure          |

## References

### Official Documentation

1. AWS - What is Cloud Computing? <https://aws.amazon.com/what-is-cloud-computing/>
2. Microsoft Azure - What is Cloud Computing? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-computing/>
3. Google Cloud - What is Cloud Computing? <https://cloud.google.com/learn/what-is-cloud-computing>
4. NIST Definition of Cloud Computing (SP 800-145) <https://csrc.nist.gov/pubs/sp/800/145/final>
5. NIST SP 800-145 PDF <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>
6. AWS - What is IaaS? <https://aws.amazon.com/what-is/iaas/>
7. Microsoft Azure - What is PaaS? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-paas/>
8. AWS - What is SaaS? <https://aws.amazon.com/what-is/saas/>

9. AWS - What is Serverless Computing?  
<https://aws.amazon.com/what-is/serverless-computing/>
11. AWS - Amazon ECS <https://aws.amazon.com/ecs/>
13. IBM - What is DRaaS? <https://www.ibm.com/think/topics/draas>
15. AWS - Machine Learning <https://aws.amazon.com/machine-learning/>
17. Microsoft Azure - What is a Private Cloud? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-private-cloud/>
19. IBM - What is Multi-Cloud? <https://www.ibm.com/think/topics/multicloud>
21. AWS - Types of Cloud Computing  
<https://aws.amazon.com/types-of-cloud-computing/>
23. IBM - IaaS, PaaS, SaaS <https://www.ibm.com/think/topics/iaas-paas-saas>
25. AWS - Containers <https://aws.amazon.com/containers/>
27. AWS EC2 Instance Types <https://aws.amazon.com/ec2/instance-types/>
29. AWS EC2 Spot Instances <https://aws.amazon.com/ec2/spot/>
31. AWS - Amazon EBS <https://aws.amazon.com/ebs/>
33. AWS S3 Storage Classes - Glacier  
<https://aws.amazon.com/s3/storage-classes/glacier/>
35. AWS - Amazon VPC <https://aws.amazon.com/vpc/>
37. AWS - Elastic Load Balancing  
<https://aws.amazon.com/elasticloadbalancing/>
39. AWS VPN <https://aws.amazon.com/vpn/>
41. AWS Route 53 <https://aws.amazon.com/route53/>
43. AWS Transit Gateway <https://aws.amazon.com/transit-gateway/>
45. AWS - Amazon DynamoDB <https://aws.amazon.com/dynamodb/>
47. AWS - Amazon Redshift <https://aws.amazon.com/redshift/>
49. AWS - Amazon Neptune <https://aws.amazon.com/neptune/>
51. AWS Pricing <https://aws.amazon.com/pricing/>
53. AWS Savings Plans <https://aws.amazon.com/savingsplans/>
55. AWS EC2 On-Demand Pricing  
<https://aws.amazon.com/ec2/pricing/on-demand/>
57. AWS IAM <https://aws.amazon.com/iam/>
59. AWS Security <https://aws.amazon.com/security/>
61. AWS VPC Security Groups  
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>
63. AWS Key Management Service <https://aws.amazon.com/kms/>
65. AWS Compliance <https://aws.amazon.com/compliance/>
67. Terraform Documentation <https://www.terraform.io/docs>
69. AWS CloudWatch <https://aws.amazon.com/cloudwatch/>
71. AWS Backup <https://aws.amazon.com/backup/>
73. AWS Tagging Best Practices  
[https://docs.aws.amazon.com/general/latest/gr/aws\\_tagging.html](https://docs.aws.amazon.com/general/latest/gr/aws_tagging.html)
75. AWS Service Level Agreements  
<https://aws.amazon.com/legal/service-level-agreements/>
77. Kubernetes Concepts <https://kubernetes.io/docs/concepts/overview/>
79. AWS SQS <https://aws.amazon.com/sqs/>
81. AWS ECR <https://aws.amazon.com/ecr/>
83. Istio Service Mesh <https://istio.io/latest/docs/overview/what-is-istio/>
85. AWS Well-Architected - Operational Excellence <https://docs.aws.amazon.com/wellarchitected/latest/operational-excellence-pillar/welcome.html>
87. AWS Well-Architected - Reliability Pillar <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>
89. AWS Well-Architected - Cost Optimization <https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html>
10. AWS - Amazon RDS <https://aws.amazon.com/rds/>
12. Firebase Documentation <https://firebase.google.com/docs>
14. Cloudflare - What is NaaS? <https://www.cloudflare.com/learning/network-layer/network-as-a-service-naas/>
16. Microsoft Azure - What is a Public Cloud? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-public-cloud/>
18. AWS - What is Hybrid Cloud?  
<https://aws.amazon.com/what-is/hybrid-cloud/>
20. Google Cloud - Sovereign Controls  
<https://cloud.google.com/sovereign-controls>
22. Microsoft Azure - What are IaaS, PaaS, and SaaS? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-are-iaas-paas-and-saas>
24. AWS - Amazon EC2 <https://aws.amazon.com/ec2/>
26. AWS Lambda <https://aws.amazon.com/lambda/>
28. AWS EC2 P5 Instances <https://aws.amazon.com/ec2/instance-types/p5/>
30. AWS - Amazon S3 <https://aws.amazon.com/s3/>
32. AWS - Amazon EFS <https://aws.amazon.com/efs/>
34. AWS EC2 Instance Storage <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html>
36. AWS VPC - Configure Subnets  
<https://docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html>
38. AWS - Amazon CloudFront <https://aws.amazon.com/cloudfront/>
40. AWS Direct Connect <https://aws.amazon.com/directconnect/>
42. AWS NAT Gateway  
<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>
44. AWS PrivateLink <https://aws.amazon.com/privatelink/>
46. AWS - Amazon ElastiCache <https://aws.amazon.com/elasticache/>
48. AWS - Amazon Timestream <https://aws.amazon.com/timestream/>
50. AWS Lake Formation <https://aws.amazon.com/lake-formation/>
52. AWS EC2 Reserved Instances  
<https://aws.amazon.com/ec2/pricing/reserved-instances/>
54. AWS Free Tier <https://aws.amazon.com/free/>
56. AWS Shared Responsibility Model  
<https://aws.amazon.com/compliance/shared-responsibility-model/>
58. AWS S3 Server-Side Encryption <https://docs.aws.amazon.com/AmazonS3/latest/userguide/serv-side-encryption.html>
60. AWS IAM - Multi-Factor Authentication  
<https://aws.amazon.com/iam/features/mfa/>
62. NIST Zero Trust Architecture  
<https://www.nist.gov/publications/zero-trust-architecture>
64. AWS WAF <https://aws.amazon.com/waf/>
66. AWS Prescriptive Guidance - Migration Strategies <https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html>
68. AWS CloudFormation <https://aws.amazon.com/cloudformation/>
70. AWS Auto Scaling <https://aws.amazon.com/autoscaling/>
72. FinOps Foundation <https://www.finops.org/>
74. AWS Control Tower <https://docs.aws.amazon.com/controltower/latest/userguide/what-is-control-tower.html>
76. AWS CodePipeline <https://aws.amazon.com/codepipeline/>
78. AWS API Gateway <https://aws.amazon.com/api-gateway/>
80. Amazon MSK <https://aws.amazon.com/msk/>
82. AWS Secrets Manager <https://aws.amazon.com/secrets-manager/>
84. AWS Well-Architected Framework  
<https://aws.amazon.com/architecture/well-architected/>
86. AWS Well-Architected - Security Pillar <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html>
88. AWS Well-Architected - Performance Efficiency <https://docs.aws.amazon.com/wellarchitected/latest/performance-efficiency-pillar/welcome.html>
90. AWS Well-Architected - Sustainability <https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html>

- 91. AWS Well-Architected Whitepaper <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- 93. AWS EC2 Availability Zones <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html>
- 95. AWS Local Zones <https://aws.amazon.com/about-aws/global-infrastructure/localzones/>
- 97. Amazon Sustainability <https://sustainability.aboutamazon.com/products-services/the-cloud>
- 99. Microsoft Azure - What is Cloud Scalability? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-cloud-scalability/>
- 92. AWS Global Infrastructure - Regions and AZs [https://aws.amazon.com/about-aws/global-infrastructure/regions\\_az/](https://aws.amazon.com/about-aws/global-infrastructure/regions_az/)
- 94. AWS CloudFront Features <https://aws.amazon.com/cloudfront/features/>
- 96. Splunk - Data Sovereignty vs Data Residency [https://www.splunk.com/en\\_us/blog/learn/data-sovereignty-vs-data-residency.html](https://www.splunk.com/en_us/blog/learn/data-sovereignty-vs-data-residency.html)
- 98. Microsoft Azure Cloud Adoption Framework <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/considerations/fundamental-concepts>
- 100. AWS Organizations <https://aws.amazon.com/organizations/>

## Technical Blogs & Tutorials

- 101. AWS Architecture Blog <https://aws.amazon.com/blogs/architecture/>
- 103. Google Cloud Blog <https://cloud.google.com/blog/>
- 105. AWS Compute Blog <https://aws.amazon.com/blogs/compute/>
- 107. Google Cloud Release Notes <https://cloud.google.com/release-notes>
- 109. Splunk - Availability Zones Guide [https://www.splunk.com/en\\_us/blog/learn/availability-zones.html](https://www.splunk.com/en_us/blog/learn/availability-zones.html)
- 111. AWS re:Invent Conference <https://reinvent.awsevents.com/>
- 113. Google Cloud Next <https://cloud.withgoogle.com/next>
- 115. Microsoft Learn for Azure <https://learn.microsoft.com/en-us/training/azure/>
- 117. AWS Best Practices for Event-Driven Architectures <https://aws.amazon.com/blogs/architecture/best-practices-for-implementing-event-driven-architectures-in-your-organization/>
- 119. Wiz - Cloud Security Best Practices <https://www.wiz.io/academy/cloud-security/cloud-security-best-practices>
- 121. IBM - 7 Rs of Cloud Migration <https://www.ibm.com/think/insights/7-rs-cloud-migration>
- 123. The Twelve-Factor App Methodology <https://12factor.net/>
- 125. VMware - DRaaS <https://www.vmware.com/topics/disaster-recovery-service-draas>
- 102. Azure Architecture Center <https://learn.microsoft.com/en-us/azure/architecture/>
- 104. AWS News Blog <https://aws.amazon.com/blogs/aws/>
- 106. Azure Updates <https://azure.microsoft.com/en-us/updates/>
- 108. IBM - What is XaaS? <https://www.ibm.com/think/topics/xaas>
- 110. Synopsys - Essential Cloud Computing Characteristics <https://www.synopsys.com/blogs/chip-design/essential-cloud-computing-characteristics.html>
- 112. Microsoft Ignite <https://ignite.microsoft.com/>
- 114. AWS Training and Certification <https://aws.amazon.com/training/>
- 116. Google Cloud Skills Boost <https://www.cloudskillsboost.google/>
- 118. OpenTelemetry Observability Primer <https://opentelemetry.io/docs/concepts/observability-primer/>
- 120. Wiz - Shared Responsibility Model <https://www.wiz.io/academy/cloud-security/shared-responsibility-model>
- 122. AWS Cost Management <https://aws.amazon.com/aws-cost-management/>
- 124. Cloudflare - Vendor Lock-In <https://www.cloudflare.com/learning/cloud/what-is-vendor-lock-in/>

## GitHub Repositories & Code Examples

- 126. AWS Samples GitHub <https://github.com/aws-samples>
- 128. Google Cloud Platform GitHub <https://github.com/GoogleCloudPlatform>
- 130. AWS CDK Examples <https://github.com/aws-samples/aws-cdk-examples>
- 132. Kubernetes Examples <https://github.com/kubernetes/examples>
- 134. Azure SDK for Python <https://github.com/Azure/azure-sdk-for-python>
- 136. Pulumi Examples <https://github.com/pulumi/examples>
- 127. Azure Samples GitHub <https://github.com/Azure-Samples>
- 129. Terraform AWS Modules <https://github.com/terraform-aws-modules>
- 131. Azure Quickstart Templates <https://github.com/Azure/azure-quickstart-templates>
- 133. AWS SDK for Python (Boto3) <https://github.com/boto/boto3>
- 135. Google Cloud Python Client <https://github.com/googleapis/google-cloud-python>
- 137. OpenTofu <https://github.com/opentofu/opentofu>

## Industry Best Practice Guides & Books

- 138. "Cloud Native Patterns" by Cornelia Davis
- 140. "The Phoenix Project" by Gene Kim, Kevin Behr, George Spafford
- 142. "Cloud Computing: Concepts, Technology & Architecture" by Thomas Erl
- 144. Google Cloud Best Practices <https://cloud.google.com/docs/enterprise/best-practices-for-enterprise-organizations>
- 146. AWS Security Best Practices <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- 148. Google Cloud Adoption Framework <https://cloud.google.com/adoption-framework>
- 150. Cloud Security Alliance <https://cloudsecurityalliance.org/>
- 152. AWS Shared Responsibility Model <https://aws.amazon.com/compliance/shared-responsibility-model/>
- 154. Google Cloud Trust Center <https://cloud.google.com/security>
- 139. "Designing Data-Intensive Applications" by Martin Kleppmann
- 141. "Site Reliability Engineering" by Google
- 143. Azure Architecture Best Practices <https://learn.microsoft.com/en-us/azure/architecture/best-practices/>
- 145. CNCF Cloud Native Glossary <https://glossary.cncf.io/>
- 147. Microsoft Cloud Adoption Framework <https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/>
- 149. AWS Cost Optimization Best Practices <https://aws.amazon.com/aws-cost-management/aws-cost-optimization/>
- 151. Cloud Native Computing Foundation <https://www.cncf.io/>
- 153. Microsoft Trust Center <https://www.microsoft.com/en-us/trust-center>
- 155. Azure Well-Architected Framework <https://learn.microsoft.com/en-us/azure/well-architected/>

156. Google Cloud Architecture Framework  
<https://cloud.google.com/architecture/framework>

## Video Resources

157. AWS Online Tech Talks <https://www.youtube.com/c/AWSOnlineTechTalks>
159. Google Cloud Tech YouTube  
<https://www.youtube.com/c/GoogleCloudTech>
161. Cloud Native Computing Foundation YouTube  
<https://www.youtube.com/c/CloudNativeComputingFoundation>
163. The Ultimate Beginner's Guide to Cloud Computing in 2026  
<https://www.youtube.com/watch?v=AOUSI1b7DAw>

## Other

164. Coursera - Cloud Computing Specializations  
<https://www.coursera.org/specializations/cloud-computing>
166. A Cloud Guru <https://acloudguru.com/>
168. Pluralsight Cloud Courses <https://www.pluralsight.com/cloud>
170. Google Kubernetes Engine  
<https://cloud.google.com/kubernetes-engine/docs>
172. Terraform Documentation <https://www.terraform.io/docs>
174. VMware Cloud  
<https://www.vmware.com/products/cloud-infrastructure.html>
176. Azure Stack <https://azure.microsoft.com/en-us/products/azure-stack/>
178. Azure Arc <https://azure.microsoft.com/en-us/products/azure-arc/>
180. AWS GovCloud <https://aws.amazon.com/govcloud-us/>
182. Amazon ECS <https://aws.amazon.com/ecs/>
184. Google Firebase <https://firebase.google.com/docs>
186. Google BigQuery <https://cloud.google.com/bigquery/docs>
188. Google Cloud SDK <https://cloud.google.com/sdk/docs>
190. AWS - Compare Block, File, Object Storage <https://aws.amazon.com/compare/the-difference-between-block-file-object-storage/>
192. IBM - Object vs File vs Block Storage  
<https://www.ibm.com/think/topics/object-vs-file-vs-block-storage>
194. AWS Disaster Recovery Shared Responsibility <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/shared-responsibility.html>
196. NIST Cloud Computing Reference Architecture (SP 500-292)  
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication500-292.pdf>
198. Google Compute Engine - Bare Metal Instances  
<https://docs.cloud.google.com/compute/docs/instances/bare-metal-instances>
200. AWS Elastic Disaster Recovery <https://aws.amazon.com/disaster-recovery/>
158. Microsoft Azure YouTube Channel  
<https://www.youtube.com/c/MicrosoftAzure>
160. AWS re:Invent Videos  
<https://www.youtube.com/user/AmazonWebServices>
162. freeCodeCamp - AWS Certified Cloud Practitioner  
<https://www.youtube.com/watch?v=SOTamWNgDKc>
165. edX - Cloud Computing Courses  
<https://www.edx.org/learn/cloud-computing>
167. Cloud Academy <https://cloudacademy.com/>
169. Docker Documentation <https://docs.docker.com/>
171. Azure Container Instances  
<https://azure.microsoft.com/en-us/products/container-instances/>
173. Azure Resource Manager Templates <https://learn.microsoft.com/en-us/azure/azure-resource-manager/templates/>
175. OpenStack Documentation <https://docs.openstack.org/>
177. AWS Outposts <https://aws.amazon.com/outposts/>
179. Google Anthos <https://cloud.google.com/anthos/docs>
181. Azure Government <https://azure.microsoft.com/en-us/explore/global-infrastructure/government/>
183. AWS Amplify <https://aws.amazon.com/amplify/>
185. Microsoft Azure SQL Database  
<https://azure.microsoft.com/en-us/products/azure-sql/database/>
187. Microsoft Azure Synapse Analytics  
<https://azure.microsoft.com/en-us/products/synapse-analytics/>
189. AWS Global Infrastructure  
<https://aws.amazon.com/about-aws/global-infrastructure/>
191. Google Cloud - Object vs Block vs File Storage  
<https://cloud.google.com/discover/object-vs-block-vs-file-storage>
193. AWS Networking Services Whitepaper <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/networking-services.html>
195. CNCF State of Cloud Native Development Q1 2026  
<https://www.cncf.io/reports/state-of-cloud-native-development-q1-2026/>
197. Azure - What are Private, Public, Hybrid Clouds? <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-are-private-public-hybrid-clouds>
199. Pulumi Documentation <https://www.pulumi.com/docs/>